

Full Stack Development

🕒 Created	February 24, 2026 1:31 AM
👤 Created by	 SHAKIL AHMAD
📁 Type	Roadmap
☑ Reviewed	☒
🕒 Last edited time	April 23, 2026 6:31 PM

▼ MODULE 0 — Quick Build Foundation

PART 1 — Setup & Web Basics

- What is web development (frontend vs backend)
 - How a website works (browser → HTML → CSS → JS)
 - Tools setup (VS Code, browser)
 - Folder & file structure
 - First HTML file run
-

PART 2 — HTML Fundamentals (Structure)

- HTML document structure (`html` , `head` , `body`)
- Headings, paragraphs, text elements
- Links (`a`) and images (`img`)
- Lists (`ul` , `ol` , `li`)
- Containers (`div` , `span`)
- Forms basics (`input` , `button` , `label`)
- Semantic tags (header, section, footer basics)

Build: Personal webpage (basic structure)

PART 3 — CSS Fundamentals (Styling)

- What is CSS & how to add it (inline, internal)
- Selectors (element, class, id)
- Colors, fonts, text styling
- Box model (margin, padding, border)
- Display (block, inline, inline-block)
- Basic styling patterns

Build: Style personal webpage

PART 4 — Layout with Flexbox

- Why layout matters
- `display: flex`
- Direction (row, column)
- Alignment (`justify-content` , `align-items`)
- Centering elements
- Basic responsive thinking

Build: Centered card layout

PART 5 — JavaScript Basics (Logic)

- What is JavaScript
- Variables (`let` , `const`)
- Data types (basic)
- Operators
- Conditions (`if/else`)
- Functions (basic usage)

Practice: Small logic examples

PART 6 — Events & Interaction

- What are events
- Click events
- Connecting JS with HTML
- Basic user interaction flow

Practice: Button click → alert

PART 7 — DOM Manipulation (Core)

- What is DOM
- Selecting elements (`querySelector`)
- Changing content (`innerText`)
- Reading input values
- Event → update UI flow

Build: Input + button → display text

PART 8 — Mini Project (To-Do App)

- Project structure setup
 - Add task functionality
 - Create dynamic elements
 - Append to list
 - Delete task functionality
 - Basic styling
-

▼ MODULE 1 — Internet, Systems, HTML & CSS Engineering

Outcome: student understands web from wire → browser → DOM → UI.

PART 1 — How Internet Actually Works

Core Topics

- What is internet physically (cables, ISP, backbone)
- Client → ISP → DNS → Server → Response flow
- IP address (IPv4 vs v6)
- Ports & sockets basics
- Domain vs hosting vs server
- Packet travel visualization
- Latency, bandwidth, ping
- CDN basics (Cloudflare concept)
- Request lifecycle overview
- How websites reach millions globally

Practical

- Trace route to google
- Ping test & latency analysis
- Use browser network tab deeply
- Draw full request diagram

Micro Project

Internet Request Visual Map

- Visual diagram tool (HTML/CSS)

- Show step-by-step request journey
-

PART 2 — DNS Deep Dive

Core Topics

- What DNS really is
- Recursive vs iterative resolution
- DNS resolver
- Root servers
- TLD servers
- Authoritative DNS
- DNS caching
- TTL concept
- DNS lookup timing
- DNS failures

Practical

- Use `nslookup`
- Use `dig`
- Trace `DNS` resolution
- Inspect `DNS` timing in browser

Mini Project

DNS Flow Visualizer

- Input domain
 - Show DNS flow steps (simulated)
 - Show resolution timeline
-

PART 3 — TCP/IP & Network Communication

Core Topics

- TCP vs UDP
- TCP 3-way handshake
- Packet transfer
- Retransmission
- Congestion control basics
- Ports & sockets
- HTTP over TCP
- HTTPS over TLS
- Network errors
- How streaming works

Practical

- Observe handshake in DevTools
- Inspect headers deeply
- Packet size observation
- Keep-alive analysis

Main Project 1

Raw TCP Server (Node net module)

- Create TCP server
 - Accept connections
 - Send manual responses
 - Log packets
 - Understand sockets
-

PART 4 — TLS, HTTPS & Security Layer

Core Topics

- Why HTTPS exists
- TLS handshake
- Certificates
- Public/private keys basics
- Encryption basics
- MITM concept
- Secure cookies
- HTTPS request lifecycle
- Browser security indicators
- HSTS basics

Practical

- Inspect HTTPS request
- Check certificates
- Use SSL checker
- Compare HTTP vs HTTPS timing

Mini Project

HTTPS Inspection Report

- Analyze 5 websites
 - Certificate details
 - Security headers
 - TLS version comparison
-

PART 5 — HTTP Protocol Deep Dive

Core Topics

- HTTP request structure
- Methods (GET/POST/PUT/PATCH/DELETE)
- Status codes deeply
- Headers deep dive
- Cookies
- Sessions basics
- HTTP1 vs HTTP2 vs HTTP3
- Persistent connections
- Caching headers
- Compression

Practical

- Use Postman/Thunder
- Send manual requests
- Modify headers
- Observe caching behavior

Main Project 2

Build Minimal HTTP Server From Scratch

(using Node `http` module)

- Routing
 - Headers
 - JSON response
 - Static file serving
 - Logging middleware
-

PART 6 — OS & Runtime Basics for Web Devs

Core Topics

- What OS actually does
- Processes vs threads
- Memory basics (stack vs heap)
- CPU scheduling basics
- How Node runs on OS
- File system basics
- Ports & processes
- Killing processes
- Environment variables
- Background services

Practical

- Use task manager/htop
- Kill ports
- Run multiple servers
- Inspect memory usage

Mini Project

System Monitor CLI (basic)

- Show memory usage
 - Show running process
 - Show port usage
-

PART 7 — Browser Internals & Rendering Pipeline

Core Topics

- How browser loads site
- HTML parsing → DOM
- CSS parsing → CSSOM
- Render tree
- Layout
- Paint
- Compositing
- Reflow & repaint
- JS blocking rendering
- Rendering optimization basics

Practical

- Chrome performance tab
- Disable CSS → observe
- Block JS → observe
- Layout shift testing

Main Project 3

Rendering Pipeline Visualizer

- Step-by-step render visualization
 - DOM → CSSOM → Paint simulation
-

PART 8 — Browser Event Loop & Runtime

Core Topics

- Call stack
- Web APIs

- Callback queue
- Microtask vs macrotask
- Event loop flow
- setTimeout vs promise
- Rendering vs JS execution
- Blocking main thread
- Memory leaks basics
- Debugging async

Practical

- Write event loop experiments
- setTimeout vs promise tests
- Block thread test
- DevTools performance recording

Mini Project

Event Loop Simulator UI

- Show task queue
 - Show call stack
 - Show execution flow
-

PART 9 — HTML Engineering & Semantic Architecture

Core Topics

- HTML parsing algorithm
- DOM tree creation
- Semantic tags deeply

- Accessibility (ARIA)
- Screen readers basics
- SEO technical foundations
- Meta tags
- Structured data (schema)
- Forms architecture
- Document structure strategy

Practical

- Accessibility audit
- Lighthouse SEO audit
- Screen reader test
- Structured data testing

Main Project 4

Accessible Multi-Page Blog

- Semantic HTML
 - Accessible navigation
 - Structured data
 - SEO optimized
-

PART 10 — Advanced HTML Systems & Forms at Scale

Core Topics

- Form architecture
- Validation patterns
- Client vs server validation

- Error UX
- Multi-step forms
- Large form performance
- Input accessibility
- Dynamic forms
- File upload basics
- HTML security basics

Final Projects (Module 1)

Project 5 — SEO Optimized Landing Page

- Perfect semantics
- Performance optimized
- Accessibility score 95+
- Structured data

Project 6 — Enterprise Dynamic Form System

- Multi-step form
- Validation system
- Error handling
- Accessible
- Clean architecture

▼ MODULE 2 — Programming & JavaScript Engineering (Core → Advanced Internals)

Goal: turn non-coder → dangerous JS engineer.

After this module: student stronger than most React devs already.

PART 1 — Programming Foundations

Core Topics

- What programming actually is
- How code executes
- Variables & data types deeply
- Operators & logic
- Control flow (if/else, loops)
- Functions basics
- Writing clean logic
- Pseudocode thinking
- Debugging mindset
- Problem-solving approach

Practical

- 15 beginner logic problems
- Flowchart → code conversion
- Debug broken programs

Mini Build

CLI Calculator (pure JS)

- Input-based
 - Multiple operations
 - Error handling
-

PART 2 — Memory Model & JS Engine Internals

Core Topics

- How JS runs inside engine (V8 concept)
- Compilation vs interpretation
- Stack vs heap deeply
- Primitive vs reference
- Memory allocation
- Garbage collection basics
- Memory leaks
- Pass by value vs reference
- Object memory behavior
- Performance impact of memory

Practical

- Memory visualization exercises
- Debug memory leak example
- Chrome memory tab usage

Mini Project

Memory Visual Demo

- Show object reference changes
 - Stack vs heap simulation UI
-

PART 3 — Execution Context & Call Stack

Core Topics

- Global execution context
- Function execution context
- Call stack behavior
- Hoisting deeply

- Temporal dead zone
- var vs let vs const internals
- Function declaration vs expression
- Strict mode
- Debugging scope errors
- Execution tracing

Practical

- Trace execution step by step
- Predict outputs
- Debug hoisting bugs

Mini Project

Execution Context Visualizer

- Call stack simulation
 - Function execution trace UI
-

PART 4 — Scope Chain & Closures (Deep)

Core Topics

- Lexical scope
- Scope chain resolution
- Closure definition deeply
- Real-world closure usage
- Private variables
- Factory functions
- Module pattern
- Closure memory behavior

- Common closure bugs
- Interview-level closure problems

Practical

- Closure experiments
- Fix closure bugs
- Write private state functions

Main Project 1

Build Private State Manager

- Hidden variables via closure
 - Controlled access methods
 - Counter system
 - Config manager
-

PART 5 — Objects, Prototypes & Inheritance

Core Topics

- Objects deeply
- this keyword mastery
- Prototype chain
- Prototypal inheritance
- Constructor functions
- Class syntax vs prototype
- Method binding
- Object.create
- ES6 classes internals
- OOP vs prototype

Practical

- Build prototype chain manually
- Convert class → prototype
- this debugging exercises

Main Project 2

Mini OOP Library System

- Class-based system
 - Prototype-based system
 - Compare both approaches
-

PART 6 — Event Loop & Async JavaScript Internals

Core Topics

- Single-threaded nature
- Call stack
- Web APIs
- Callback queue
- Microtask vs macrotask
- Event loop cycle
- setTimeout behavior
- Promise queue priority
- Blocking main thread
- Async debugging

Practical

- Event loop experiments
- Predict output tests

- Blocking thread tests
- DevTools async debugging

Mini Project

Event Loop Visual Simulator

- Task queue visualization
 - Promise vs timeout flow
 - Execution order display
-

PART 7 — Promises & Async/Await Deep Dive

Core Topics

- Callback hell
- Promise states
- Promise chaining
- Error propagation
- finally()
- Async/await internally
- Parallel vs sequential async
- Promise.all / race / allSettled
- Retry logic
- Writing clean async code

Practical

- Convert callbacks → promises
- Build retry logic
- Parallel API calls
- Error handling patterns

Main Project 3

Build Promise From Scratch

Implement:

- then()
- catch()
- finally()
- chaining
- resolve/reject handling

(One of most important builds)

PART 8 — Functional Programming in JS

Core Topics

- Pure functions
- Immutability
- Higher-order functions
- map/filter/reduce deeply
- Function composition
- Currying
- Memoization
- Functional utilities
- Avoid side effects
- FP vs OOP usage

Practical

- Write custom map/filter/reduce
- Memoization exercises

- Compose functions
- Build FP utilities

Mini Project

Functional Utility Toolkit

- `compose()`
 - `pipe()`
 - `memoize()`
 - `debounce()`
 - `throttle()`
-

PART 9 — Error Handling & Code Quality Architecture

Core Topics

- `try/catch` deeply
- Throwing custom errors
- Error propagation
- Async error handling
- Global error handling
- Writing safe functions
- Defensive programming
- Logging basics
- Clean code principles
- Debugging mindset

Practical

- Write safe wrappers

- Handle async errors
- Debug broken app
- Add logging layer

Mini Project

Error Handling Framework

- Custom error classes
 - Global handler
 - Logging system
 - Safe execution wrapper
-

PART 10 — Core Engineering Projects (Module Finale)

Main Project 4 — Custom Event Emitter (Like Node.js)

Build from scratch:

- on()
 - emit()
 - once()
 - off()
 - Event queue
 - Async support
-

Main Project 5 — JavaScript Utility Library (Like Lodash Mini)

Implement:

- deepClone
- debounce
- throttle

- memoize
 - curry
 - compose
 - flatten array
 - unique array
 - custom map/filter/reduce
-

Main Project 6 — 50 Logic Problems Pack

Cover:

- Strings
 - Arrays
 - Objects
 - Recursion
 - Async problems
 - Real-world logic
-

▼ MODULE 3 — Terminal, Git & Engineering Workflow (Deep Engineer Mode)

Goal: make learner dangerous with system, terminal, and Git.

After this: works like real backend/full-stack engineer, not tutorial kid.

PART 1 — Operating System Fundamentals for Developers

Core Topics

- What OS actually does
- Kernel vs user space
- Process lifecycle
- Threads vs processes
- Scheduling basics
- Memory allocation basics
- Virtual memory concept
- File descriptors
- How programs run
- How Node/Python run on OS

Practical

- Observe running processes
- Track memory usage
- Kill process by PID
- Monitor CPU usage

Mini Lab

Draw full lifecycle:



run node app → OS → CPU → memory → output

PART 2 — Linux File System Internals

Core Topics

- Linux directory structure (/bin, /etc, /var)
- File system hierarchy standard

- Inodes concept
- File permissions deeply
- chmod numeric system
- chown
- Symbolic links
- Hard links
- Hidden files
- Disk usage analysis

Practical

- Create users & permissions
- Simulate restricted folders
- Create symlink system
- Disk usage audit

Mini Project

Dev Workspace Setup Script

- Auto create project folders
 - Permission setup
 - Env setup
-

PART 3 — Advanced Terminal Mastery

Core Topics

- Bash vs zsh
- Shell working
- PATH variable deeply
- Aliases & functions

- Environment variables
- Command chaining
- Pipes & redirection
- grep/awk/sed basics
- History search
- Productivity tricks

Practical

- Create custom aliases
- Build command pipelines
- Search logs via grep
- Create env configs

Mini Project

Developer Productivity Terminal Setup

- Custom aliases
 - Auto project creator
 - Git shortcuts
-

PART 4 — Bash Scripting (Automation Level)

Core Topics

- Bash script structure
- Variables & input
- Conditions & loops
- Functions in bash
- Reading files
- Writing logs

- Cron jobs
- Automating builds
- Deployment scripts
- Error handling in bash

Practical

- Write automation scripts
- Backup script
- Auto project setup script
- Server monitoring script

Main Project 1

Full Project Bootstrap Script

- Creates folder structure
 - Installs dependencies
 - Initializes git
 - Creates env files
-

PART 5 — Processes, Ports & System Networking

Core Topics

- What is a process internally
- Foreground vs background
- Process tree
- Ports & sockets
- Checking port usage
- Killing ports
- Network tools (netstat, lsof)

- Process monitoring
- Resource leaks
- Debugging stuck apps

Practical

- Find process on port
- Kill zombie process
- Monitor CPU-heavy app
- Simulate crash & debug

Mini Project

System Debug Toolkit

CLI showing:

- Running ports
 - Active processes
 - Memory usage
-

PART 6 — SSH & Remote Server Engineering

Core Topics

- What SSH actually does
- Key-based authentication
- SSH handshake basics
- Public/private keys
- Config file
- Remote command execution
- Secure file transfer (scp/rsync)
- SSH tunneling basics

- Remote dev workflow
- Server hardening basics

Practical

- Create SSH keys
- Connect remote server
- Setup passwordless login
- Transfer files
- Remote command run

Main Project 2

Remote Server Setup

- Setup VPS
 - Install Node
 - Deploy test app
 - Secure SSH login
-

PART 7 — Git Internals (Real Understanding)

Core Topics

- Why Git exists
- Git architecture
- Commit objects
- Tree objects
- Blob objects
- HEAD pointer
- Staging area

- Git lifecycle
- How commits stored
- Git vs GitHub

Practical

- Inspect .git folder
- View commit objects
- Trace commit tree
- Reset & recover files

Mini Lab

Manually recreate repo lifecycle diagram.

PART 8 — Branching, Rebasing & History Control

Core Topics

- Branching strategies (real industry)
- Feature branching
- Git flow vs trunk-based
- Merge vs rebase deeply
- Interactive rebase
- Cherry-pick
- Squash commits
- Clean commit history
- Recover deleted commits
- Undo mistakes safely

Practical

- Simulate team workflow
- Complex rebase
- Resolve history conflicts
- Clean messy repo

Main Project 3

Team Git Simulation

- 3 feature branches
 - Merge conflicts
 - Rebase workflow
 - Release branch simulation
-

PART 9 — Conflict Resolution & Advanced Git Tools

Core Topics

- Merge conflicts deeply
- Conflict markers
- Visual merge tools
- Stash internals
- Reflog
- Bisect debugging
- Tagging versions
- Submodules basics
- Debugging broken repo
- Large repo handling

Practical

- Resolve complex conflicts

- Recover deleted branch
- Use git bisect
- Tag releases

Mini Project

Broken Repo Recovery Challenge

- Given corrupted repo
 - Restore working state
 - Fix history
-

PART 10 — Git Hooks, Automation & Final Projects

Core Topics

- Git hooks internally
 - Pre-commit hooks
 - Husky setup
 - Lint-staged
 - Commit standards
 - Automated formatting
 - CI hooks intro
 - Release automation
 - Team workflow automation
 - Dev workflow optimization
-

FINAL PROJECTS (Module 3)

Project 1 — CLI Automation Tool

Build powerful CLI:

- Create project
 - Setup env
 - Initialize git
 - Install deps
 - Start dev server
-

Project 2 — Complete Team Git Workflow Simulation

Simulate:

- 5 developers
 - Feature branches
 - PR workflow
 - Release cycle
 - Hotfix branch
 - Version tagging
-

Project 3 — Remote Production Server Setup

- Buy VPS
 - SSH secure setup
 - Deploy backend
 - Setup logs
 - Setup restart script
-

▼ MODULE 4 — Backend Engineering I (Node.js + REST + MongoDB + PostgreSQL)

Goal: turn learner into real backend developer (not CRUD tutorial guy).

Outcome: can design, build, secure, and structure production-grade APIs.

PART 1 — Node.js Runtime Internals (Beyond “console.log”)

Core Topics

- How Node works (V8 + libuv)
- Event loop phases deeply
- Microtasks vs macrotasks
- Thread pool
- Non-blocking I/O
- Streams concept
- Buffers
- Backpressure
- Worker threads intro
- Blocking vs non-blocking patterns

Practical

- Create CPU blocking example
- Compare sync vs async file read
- Build stream-based file reader
- Analyze event loop delay

Mini Project

File Streaming Server

- Stream large file
- Monitor memory usage

- Implement backpressure handling
-

PART 2 — Project Architecture & Clean Backend Structure

Core Topics

- MVC vs layered architecture
- Controller-service-repository pattern
- Separation of concerns
- Folder structuring
- Config management
- Environment variables
- Dependency management
- Logging strategy intro
- API versioning strategy
- Modular code design

Practical

- Refactor messy project
- Setup config system
- Create base folder template

Mini Build

Backend Boilerplate Setup

- Clean folder structure
 - Central config
 - Logger base
 - Global error handler skeleton
-

PART 3 — Express Deep Dive & Middleware Architecture

Core Topics

- How Express works internally
- Middleware chain execution
- Custom middleware writing
- Error middleware
- Request lifecycle
- Route grouping
- Router modularization
- Async middleware patterns
- Handling async errors properly
- Performance considerations

Practical

- Write auth middleware
- Write request logger
- Create rate limiter middleware
- Implement error middleware

Main Build

Production Express Core

- Modular routes
 - Centralized error handling
 - Middleware layering
 - Request logging
-

PART 4 — REST API Engineering (Professional Level)

Core Topics

- REST principles deeply
- Resource naming standards
- Proper HTTP methods
- Status codes mastery
- Idempotency
- Pagination strategies
- Filtering & sorting
- API versioning
- Consistent response format
- API documentation intro

Practical

- Design resource map
- Implement filter/sort query params
- Implement pagination logic
- Standardize API response

Main Project 1

Production REST API

- 4 entities
- Full CRUD
- Pagination
- Filtering
- Validation

- Structured response format
-

PART 5 — MongoDB Deep Dive (NoSQL Engineering)

Core Topics

- NoSQL fundamentals
- BSON structure
- Document modeling strategies
- Embedding vs referencing
- Schema design trade-offs
- Indexing in MongoDB
- Compound indexes
- Aggregation pipeline intro
- Query optimization basics
- Mongoose vs native driver

Practical

- Design scalable schema
- Add indexes
- Run performance comparison
- Write aggregation queries

Mini Project

MongoDB Analytics Module

- Aggregation for reports
 - Indexed queries
 - Performance comparison
-

PART 6 — PostgreSQL Fundamentals (SQL Engineering)

Core Topics

- Relational database fundamentals
- Tables, rows, constraints
- Primary/foreign keys
- Normalization (1NF–3NF)
- Indexing (B-tree concept)
- Query planning basics
- JOINS deeply
- Transactions & ACID
- Isolation levels intro
- Performance fundamentals

Practical

- Design normalized schema
- Write complex JOIN queries
- Add index & compare performance
- Run transaction example

Mini Project

Relational Data Module

- Multi-table relations
 - Transaction-safe operation
 - Indexed queries benchmark
-

PART 7 — Prisma ORM & Data Layer Engineering

Core Topics

- What ORM actually does
- Prisma schema modeling
- Relations in Prisma
- Migrations deeply
- Seeding database
- Transactions in Prisma
- Query optimization
- Avoiding N+1 problem
- Raw queries vs ORM
- Data access layer separation

Practical

- Setup Prisma
- Design relational schema
- Run migrations
- Build repository layer

Main Build

Structured Data Access Layer

- Repository pattern
- Typed queries
- Transaction support

PART 8 — Authentication & Security Basics (JWT Level)

Core Topics

- What authentication really is
- Sessions vs JWT
- JWT structure (header, payload, signature)
- Access vs refresh tokens
- Password hashing (bcrypt concept)
- Token expiration
- Secure middleware
- Protecting routes
- Role-based access basics
- Security pitfalls

Practical

- Hash passwords
- Create JWT system
- Protect routes
- Add role-based middleware

Main Project 2

JWT Auth System

- Register/Login
 - Token generation
 - Route protection
 - Role-based authorization
-

PART 9 — Error Handling, Validation & API Standardization

Core Topics

- Global error architecture
- Custom error classes
- Async error wrapper
- Validation layer (Zod/Joi)
- Input sanitization
- Standard error response format
- Logging errors
- Handling database errors
- Production-safe error messages
- Defensive backend coding

Practical

- Implement validation middleware
- Create custom error types
- Central error handler
- Log structured errors

Mini Project

Error Handling Framework

- Standardized API responses
 - Validation layer
 - Safe error outputs
-

PART 10 — Final Production Build (Module Finale)

Combine everything.

Final Project A — Production-Ready REST API

Includes:

- MongoDB + PostgreSQL integration
 - Auth system
 - Pagination & filtering
 - Index optimization
 - Clean architecture
 - Error handling
 - Environment configs
-

Final Project B — API Performance Optimization

- Add indexes
 - Benchmark queries
 - Compare before/after
 - Measure response times
 - Optimize slow endpoints
-

Final Project C — API Documentation

- Document endpoints
 - Define response formats
 - Add sample requests
 - Prepare dev-ready docs
-

▼ MODULE 5 — React Engineering I (Core → Advanced Patterns)

Goal: turn learner into production-level React engineer.

Not “tutorial React dev”. Real scalable frontend builder.

PART 1 — React Foundations & Rendering Mindset

Core Topics

- Why React exists (SPA problem)
- Virtual DOM vs real DOM
- Rendering cycle
- Reconciliation
- CSR vs SSR basics
- Component-based architecture thinking
- JSX deeply
- Props system
- Component reusability mindset
- Clean component design

Practical

- Break UI into components
- Render dynamic data
- Build nested components

Mini Build

Componentized Static Website

- Navbar
- Sidebar
- Cards
- Footer

All reusable.

PART 2 — Modern Hooks Mastery (Deep)

Core Topics

- useState deep behavior
- useEffect lifecycle truth
- Dependency array mastery
- useRef deeply
- useMemo correctly
- useCallback correctly
- Custom hooks architecture
- Avoiding re-render traps
- Hook composition patterns
- Hook debugging

Practical

- Build custom hooks
- Fix infinite re-renders
- Optimize heavy components

Mini Project

Custom Hooks Toolkit

- useDebounce
 - useToggle
 - useFetch
 - useLocalStorage
 - usePrevious
-

PART 3 — Component Architecture (Senior-Level Thinking)

Core Topics

- Smart vs dumb components
- Container pattern
- Presentational pattern
- Controlled vs uncontrolled components
- Compound components
- Headless component concept
- Lifting state correctly
- Avoiding prop drilling
- Context usage properly
- Folder structuring

Practical

- Build reusable modal system
- Build reusable dropdown
- Build tab system

Main Build

Reusable UI Component System

- Modal
 - Tabs
 - Dropdown
 - Toast system
 - Table component
-

PART 4 — State Management (Zustand + Redux Toolkit)

Core Topics

- Local vs global state
- When to use global state
- Zustand deep usage
- Store design
- Redux Toolkit modern usage
- Slice architecture
- Async thunk
- RTK Query basics
- Normalized state
- State debugging

Practical

- Build Zustand store
- Convert to Redux store
- Handle async actions
- Debug state flow

Mini Project

Global App State System

- Auth store
- Theme store
- Notification store
- Cart/dashboard state

PART 5 — Server State Management (TanStack Query Deep)

Core Topics

- Server vs client state
- Why TanStack Query
- Query lifecycle
- Caching deeply
- Refetch strategies
- Optimistic updates
- Pagination queries
- Infinite scroll queries
- Mutation handling
- Error handling

Practical

- Fetch APIs with caching
- Build optimistic update UI
- Handle API errors properly
- Add pagination fetching

Mini Project

API Data Layer System

- Cached API layer
- Auto refetch
- Mutation updates
- Error fallback UI

PART 6 — Form Engineering (Enterprise Level)

Core Topics

- Controlled vs uncontrolled deeply
- React Hook Form deep
- Zod validation
- Schema-based validation
- Dynamic forms
- Multi-step forms
- File upload forms
- Error UX patterns
- Form performance optimization
- Accessibility in forms

Practical

- Build login/signup forms
- Dynamic field forms
- Validation system
- Multi-step flow

Main Project 1

Enterprise Multi-Step Form

- 6–8 steps
- Validation per step
- Save progress
- File uploads
- Error handling
- Clean UX

PART 7 — Performance Optimization (Serious Level)

Core Topics

- Re-render causes
- React rendering behavior
- Memoization strategy
- useMemo vs useCallback
- React.memo properly
- Lazy loading components
- Code splitting
- Bundle analysis
- Virtualized lists
- Performance debugging

Practical

- Optimize slow UI
- Implement lazy loading
- Analyze bundle size
- Reduce renders

Mini Project

Performance Refactor Lab

- Given slow dashboard
 - Optimize to fast
 - Reduce renders by 60%+
-

PART 8 — Reusable Systems & Design Patterns

Core Topics

- Reusable component strategy

- Design systems basics
- Atomic design
- Theming systems
- Dark/light mode architecture
- Role-based UI rendering
- Permission-based UI
- Layout systems
- Plugin-based UI idea
- Clean architecture frontend

Practical

- Build theme switcher
- Role-based sidebar
- Layout system
- Component config system

Mini Project

Mini Design System

- Buttons
 - Inputs
 - Cards
 - Theme system
 - Layout wrappers
-

PART 9 — Code Splitting, Routing & Large App Structure

Core Topics

- React Router deep
- Nested routes
- Layout routing
- Protected routes
- Lazy loading routes
- Code splitting strategy
- Suspense properly
- Folder structuring for large apps
- Feature-based architecture
- Maintainability patterns

Practical

- Setup full routing system
- Lazy load pages
- Protected routes
- Role-based routing

Mini Project

Scalable App Structure

- Auth routes
 - Dashboard routes
 - Admin routes
 - Lazy modules
-

PART 10 — Final Production Projects (Module 5)

Project 1 — Reusable Component Library

Includes:

- Buttons
 - Inputs
 - Modal
 - Table
 - Form components
 - Theme support
 - Fully reusable
-

Project 2 — Multi-Step Enterprise Form System

Includes:

- Dynamic fields
 - Validation
 - File upload
 - Error handling
 - Step navigation
 - Save progress
-

Project 3 — Role-Based Dashboard Frontend

Includes:

- Auth UI
 - Sidebar by role
 - API integration
 - Charts/cards
 - Global state (Zustand/Redux)
 - TanStack Query
 - Optimized performance
-

▼ MODULE 6 — Backend Engineering II

Goal: make backend **production-grade & scalable**.

After this: learner can build systems used by thousands+ users.

PART 1 — Redis Deep Dive (Not Just Key-Value)

Core Topics

- What Redis actually is
- In-memory storage concept
- Redis architecture
- Data structures:
 - Strings
 - Lists
 - Sets
 - Sorted sets
 - Hashes
- TTL & expiration
- Persistence (RDB vs AOF)
- Pub/Sub basics
- Redis vs DB difference
- When NOT to use Redis

Practical

- Setup Redis locally
- Store/retrieve data
- TTL testing
- Benchmark Redis vs DB

Mini Project

Redis Playground API

- Store sessions
 - Cache responses
 - TTL based keys
 - Leaderboard via sorted set
-

PART 2 — Caching Architecture (Real Production Thinking)

Core Topics

- Why caching matters
- Types of caching:
 - Memory cache
 - Redis cache
 - CDN cache
- Cache aside pattern
- Write-through cache
- Write-back cache
- Cache invalidation
- Stale data problems
- Cache TTL strategies
- Performance benchmarking

Practical

- Cache API responses
- Compare cached vs uncached

- Implement cache invalidation
- Measure response time

Main Project 1

Redis Caching Layer

- Cache GET APIs
 - Invalidate on update
 - TTL strategy
 - Performance comparison
-

PART 3 — Rate Limiting & API Protection

Core Topics

- Why rate limiting matters
- Brute force prevention
- IP-based limiting
- Token-based limiting
- Sliding window algorithm
- Fixed window
- Distributed rate limiting
- Redis-based limiter
- API throttling strategies
- Handling abuse

Practical

- Implement rate limiter middleware
- Redis rate limiting
- Simulate attack traffic

- Block abusive clients

Mini Project

API Protection Layer

- Rate limiting
 - Request logging
 - Abuse detection
 - Temporary blocking
-

PART 4 — Secure API Design & Backend Security

Core Topics

- API security mindset
- Input validation deeply
- Sanitization
- SQL/NoSQL injection prevention
- XSS basics backend
- CORS deep configuration
- Helmet security headers
- Secure cookies
- Token security
- Secrets management (.env safety)

Practical

- Attack simulation
- Secure endpoints
- Add validation layer
- Secure headers

Main Build

Secure Backend Layer

- Validation middleware
 - Sanitization
 - Security headers
 - Protected endpoints
-

PART 5 — File Upload Systems (Production Level)

Core Topics

- Multipart form data
- File upload flow
- Storage strategies:
 - Local storage
 - Cloud storage (concept)
- File validation
- File size limits
- Secure upload handling
- Image processing basics
- Streaming uploads
- Serving files securely
- Signed URLs concept

Practical

- Upload images
- Validate file types
- Resize images

- Serve uploaded files

Mini Project

File Upload Service

- Profile image upload
 - Validation
 - Storage
 - Retrieval API
-

PART 6 — WebSockets & Real-Time Systems

Core Topics

- HTTP vs WebSocket
- WebSocket handshake
- Persistent connection
- Real-time architecture
- Rooms & channels
- Broadcasting
- Presence system
- Scaling socket servers
- Redis pub/sub for scaling
- Handling disconnects

Practical

- Create WebSocket server
- Join rooms
- Broadcast messages
- Handle reconnects

Main Project 2

Real-Time Chat Backend

- WebSocket server
 - Join rooms
 - Message broadcast
 - User presence
 - Typing indicator
 - Basic persistence
-

PART 7 — Background Jobs & Queues (BullMQ)

Core Topics

- Why background jobs
- Queue systems
- Job lifecycle
- Redis queues
- BullMQ deep usage
- Workers
- Retry strategies
- Delayed jobs
- Cron jobs
- Failure handling

Practical

- Setup BullMQ
- Process background jobs
- Retry failed jobs

- Schedule tasks

Main Project 3

Background Job Processing System

- Email queue
 - Image processing queue
 - Retry failed jobs
 - Job dashboard basics
-

PART 8 — Logging & Monitoring Architecture

Core Topics

- Why logging matters
- Types of logs
- Request logging
- Error logging
- Structured logging
- Log levels
- Centralized logging concept
- Monitoring basics
- Debugging production issues
- Observability intro

Practical

- Create logger system
- Log API requests
- Log errors
- Track response times

Mini Project

Logging Middleware System

- Request logs
 - Error logs
 - Performance logs
 - Log formatting
-

PART 9 — API Throttling & Production Hardening

Core Topics

- Throttling vs rate limit
- Burst handling
- Circuit breaker concept
- Timeout handling
- Retry strategy
- Graceful shutdown
- Memory leak detection
- Handling crashes
- Environment configs
- Production safety checklist

Practical

- Add timeout handling
- Simulate heavy load
- Crash recovery testing
- Graceful shutdown setup

Mini Project

Production Hardening Layer

- Rate limit
 - Timeout handling
 - Graceful shutdown
 - Error fallback
-

PART 10 — Final Integration Projects

Final Project 1 — Real-Time Chat Backend (Full)

Includes:

- WebSockets
 - Redis pub/sub
 - Auth integration
 - Message storage
 - Rate limiting
 - Logging
-

Final Project 2 — Redis Caching API Layer

Includes:

- Cached endpoints
 - Cache invalidation
 - Performance comparison
 - Logging
-

Final Project 3 — Background Job System

Includes:

- Job queue

- Email worker
 - Retry logic
 - Scheduled jobs
-

▼ MODULE 7 — Next.js

Goal: make them full-stack React architect.

After this: can build scalable SaaS with clean multi-package architecture.

PART 1 — Next.js Architecture & App Router Deep

Core Topics

- Why Next.js exists
- CSR vs SSR vs SSG vs ISR
- App Router architecture
- File-system routing deep
- Nested layouts
- Route groups
- Dynamic segments
- Metadata API
- Error boundaries
- Loading states

Practical

- Multi-layout SaaS skeleton
- Dynamic routing example
- Route-level loading UI

Mini Build

Multi-layout App Setup

- Public layout
 - Dashboard layout
 - Admin layout
-

PART 2 — React Server Components (RSC) Internals

Core Topics

- What RSC really is
- Client vs Server components
- Why RSC improves performance
- Server rendering flow
- RSC payload concept
- When NOT to use RSC
- Mixing client/server components
- Avoiding hydration mismatch
- Debugging RSC
- Performance comparison

Practical

- Convert client → server component
- Fetch data inside server component
- Compare bundle sizes

Mini Project

Hybrid Rendering Demo

- Server rendered data
- Client interactive widgets

- Minimal JS bundle
-

PART 3 — Data Fetching & Streaming

Core Topics

- Fetch in server components
- Async server components
- Suspense deeply
- Streaming UI
- Partial rendering
- Loading boundaries
- Error boundaries
- Parallel data fetching
- Waterfall problem
- Performance measurement

Practical

- Streaming dashboard data
- Parallel fetching
- Loading skeleton system

Main Project 1

Streaming UI Application

- Multiple async sections
 - Partial loading
 - Real-time updates
 - Error fallbacks
-

PART 4 — Server Actions (2026 Full Power)

Core Topics

- What server actions are
- Form handling without API routes
- Mutations from server
- Security implications
- Input validation
- Optimistic updates
- Error handling in server actions
- Transaction usage
- Performance benefits
- Server actions vs REST

Practical

- Build form with server action
- Validate inputs
- Perform DB mutation
- Add optimistic UI

Mini Project

Server Action Form System

- Create/update/delete
 - Form validation
 - Optimistic UI
-

PART 5 — Auth Integration (Production Level)

Core Topics

- Auth flow in Next
- Cookies vs JWT
- Secure cookie storage
- Middleware protection
- Role-based routing
- Server-side auth validation
- Refresh flow
- Protecting RSC
- Auth in monorepo
- Preventing token leaks

Practical

- Login/logout flow
- Protect routes
- Role-based layout
- Middleware setup

Main Project 2

Protected Multi-Role System

- User roles
- Protected dashboard
- Role-based navigation
- Secure server validation

PART 6 — API Routes & Backend Integration

Core Topics

- Route handlers deep
- REST in Next
- Using Prisma
- Validation layer
- Error handling
- File upload handling
- Background job integration
- Rate limiting integration
- API structure best practices
- Clean folder separation

Practical

- Build API route
- Connect DB
- Validate input
- Add auth middleware

Mini Build

Structured API Layer

- Modular route handlers
- Validation middleware
- Error handler

PART 7 — Caching, ISR & Performance Engineering

Core Topics

- Static rendering
- ISR deeply

- Revalidation strategies
- Cache tags
- fetch caching options
- Dynamic vs static routes
- Edge caching concept
- Reducing TTFB
- Performance benchmarking
- Production tuning

Practical

- Implement ISR
- Compare cache vs no cache
- Revalidate data
- Measure performance

Mini Project

High-Performance Blog System

- Static pages
 - Incremental updates
 - Fast load time
-

PART 8 — Edge Runtime & Advanced Deployment

Core Topics

- What Edge runtime is
- Node vs Edge differences
- Running code at edge
- Limitations of edge

- Middleware at edge
- Geolocation-based routing
- Performance trade-offs
- Security at edge
- Global scaling basics
- Edge caching strategy

Practical

- Deploy edge middleware
- Location-based content
- Compare response times

Mini Project

Edge Middleware Demo

- Geo-based routing
- Rate limit at edge
- Middleware logging

PART 9 — Monorepo Architecture (TurboRepo / PNPM Workspaces)

Core Topics

- What monorepo is
- Why monorepo matters
- PNPM workspaces deep
- TurboRepo basics
- Shared packages
- Shared UI library

- Shared types
- Backend + frontend in one repo
- Build caching
- CI strategy in monorepo

Practical

- Setup monorepo
- Create shared UI package
- Create shared types package
- Connect frontend + backend

Main Project 3

Monorepo Full-Stack Setup

- apps/web
 - apps/api
 - packages/ui
 - packages/config
 - Shared types
 - Shared validation
-

PART 10 — Final SaaS Integration Project

Combine everything.

Final Project — Full SaaS Dashboard

Includes:

- App Router architecture
- Server components
- Server actions

- Auth system
- Role-based routing
- Prisma integration
- Streaming UI
- ISR caching
- Monorepo architecture
- Optimized build

Must include:

- Dashboard
 - Admin panel
 - User profile
 - Analytics page
 - Role-based access
 - Optimized performance
-

▼ **MODULE 8 — Advanced Frontend & Creative Engineering**

Goal: build rare frontend engineer who understands animation + GPU + rendering pipeline.

After this: can build award-level interactive websites and 3D apps.

PART 1 — Animation Engineering Fundamentals

Core Topics

- Browser rendering vs animation
- `requestAnimationFrame`

- CSS vs JS animation tradeoffs
- Frame budget (16.6ms rule)
- Jank causes
- GPU vs CPU rendering
- Transform vs layout animations
- Compositing layers
- Measuring FPS
- Animation debugging tools

Practical

- Create smooth animation loop
- Measure frame drops
- Compare transform vs top/left animation

Mini Build

FPS Performance Lab

- Render 100 animated elements
 - Optimize until 60fps stable
-

PART 2 — GSAP Deep Dive (Professional Motion System)

Core Topics

- GSAP core engine
- Timelines deeply
- Stagger animations
- Easing curves
- Sequencing complex timelines

- Reusable animation modules
- GSAP context
- Killing animations safely
- Performance optimization
- Debugging animation flow

Practical

- Build timeline-based hero section
- Create reusable animation hook
- Sequence multi-step animation

Mini Project

Advanced Hero Animation System

- Multi-layer intro animation
 - Text stagger
 - Image reveal
 - Timeline orchestration
-

PART 3 — ScrollTrigger & Scroll-Based Systems

Core Topics

- ScrollTrigger internals
- Scroll-based timeline control
- Pinning sections
- Scrub animations
- Scroll velocity effects
- Horizontal scroll simulation
- Parallax systems

- Performance pitfalls
- IntersectionObserver comparison
- Smooth scrolling strategies

Practical

- Build scroll-based reveal
- Pin storytelling section
- Create horizontal scroll gallery

Main Project 1

Animated Storytelling Website

- Scroll-driven animation
 - Section pinning
 - Parallax effects
 - Optimized for performance
-

PART 4 — WebGL & GPU Pipeline Fundamentals

Core Topics

- What WebGL actually is
- GPU rendering pipeline
- Vertex shader concept
- Fragment shader concept
- Buffers
- Textures
- Draw calls
- Coordinate systems
- Camera projection basics

- Why WebGL is fast

Practical

- Render basic WebGL triangle
- Inspect shader code
- Modify fragment color dynamically

Mini Lab

Visual diagram of GPU pipeline.

PART 5 — Three.js Internals (Beyond Tutorial Level)

Core Topics

- Scene, camera, renderer architecture
- Perspective vs orthographic camera
- Geometry & buffer geometry
- Materials deeply
- Lighting system
- Shadows performance
- Orbit controls
- Raycasting
- Post-processing (EffectComposer)
- Memory disposal & cleanup

Practical

- Build 3D scene
- Add lighting & shadows
- Add controls

- Raycast object interaction

Mini Project

Interactive 3D Scene

- Multiple objects
 - Hover effects
 - Camera transitions
-

PART 6 — Shaders Introduction (GLSL Basics)

Core Topics

- What shaders actually are
- GLSL basics
- Vertex shader structure
- Fragment shader structure
- Uniforms & attributes
- Animating with time uniform
- Noise basics
- Shader performance
- Custom material creation
- When to use shaders

Practical

- Modify fragment color
- Animate wave effect
- Create simple gradient shader

Mini Project

Custom Shader Effect

- Animated background
 - Time-based color shifts
 - Interactive effect
-

PART 7 — React Three Fiber (R3F)

Core Topics

- What R3F is
- Declarative 3D
- JSX for 3D
- useFrame
- Drei utilities
- Suspense for models
- Physics integration concept
- Performance optimization
- Mixing DOM + WebGL
- Cleanup patterns

Practical

- Build R3F scene
- Load GLTF model
- Animate with useFrame
- Add environment lighting

Main Project 2

3D Product Viewer

- Load 3D model

- Rotate/zoom
 - Change colors/materials
 - Smooth animations
 - Optimized rendering
-

PART 8 — Advanced Animation Systems & Interaction Design

Core Topics

- Micro-interactions
- Gesture handling
- Mouse-based animations
- Scroll-based 3D transitions
- UI + 3D hybrid layout
- Page transition animations
- Animation architecture patterns
- Performance budgeting
- Debounce & throttle in animations
- Clean animation abstraction

Practical

- Cursor-follow effect
- Page transition animation
- Interactive hover distortions

Mini Project

Advanced Portfolio Interaction System

- Custom cursor

- Animated transitions
 - Smooth scroll
 - Micro-interactions
-

PART 9 — Performance Tuning for 3D & Animation

Core Topics

- Reducing draw calls
- Instancing
- Geometry optimization
- Texture optimization
- Lazy loading 3D models
- Memory cleanup
- Avoiding re-renders in R3F
- FPS monitoring
- Performance debugging
- Mobile optimization

Practical

- Optimize heavy scene
- Reduce polygon count
- Improve FPS from 30 → 60+

Mini Lab

Performance before vs after comparison.

PART 10 — Final Creative Engineering Projects

Project 1 — 3D Product Viewer (Production Level)

Includes:

- GLTF model loading
 - Material switching
 - Camera controls
 - Smooth transitions
 - Performance optimized
 - Mobile responsive
-

Project 2 — Animated Storytelling Website

Includes:

- ScrollTrigger timeline
 - Section pinning
 - Parallax
 - Optimized animations
 - Smooth transitions
-

Project 3 — Interactive WebGL Mini Game

Includes:

- Basic physics
 - Score system
 - 3D scene
 - Animation loop
 - Optimized performance
 - Responsive UI overlay
-

▼ MODULE 9 — Advanced Backend & Scaling Systems

Goal: turn backend dev → system-level engineer.

After this: can design scalable backend used by startups/products.

PART 1 — Scaling Mindset & System Evolution

Core Topics

- Monolith vs distributed systems
- When monolith breaks
- Scaling problems:
 - CPU
 - DB
 - latency
- Vertical vs horizontal scaling
- Stateless vs stateful services
- Bottleneck identification
- CAP theorem intro
- Consistency vs availability
- Throughput vs latency
- Real-world system examples

Practical

- Analyze scaling failure case
- Identify bottlenecks
- Design scaling plan

Mini Lab

Draw architecture:

"1k users → 1M users transition"

PART 2 — GraphQL Deep Engineering

Core Topics

- REST vs GraphQL deeply
- GraphQL architecture
- Schema design
- Types & resolvers
- Queries vs mutations
- Nested queries
- N+1 problem
- DataLoader pattern
- Pagination in GraphQL
- Error handling

Practical

- Build schema
- Create resolvers
- Optimize N+1 queries
- Add pagination

Main Project 1

Production GraphQL API

- Auth
- CRUD

- Pagination
 - Optimized resolvers
 - Error handling
-

PART 3 — API Gateway & Service Communication

Core Topics

- What API gateway is
- Why gateway needed
- Central auth handling
- Rate limiting at gateway
- Request routing
- Aggregating services
- Gateway vs BFF pattern
- Internal vs external APIs
- Service discovery basics
- Gateway logging

Practical

- Build mini API gateway
- Route to multiple services
- Central auth middleware
- Request logging

Mini Project

Node API Gateway

- Route requests
- Validate token

- Forward to services
-

PART 4 — Microservices Architecture (Real Understanding)

Core Topics

- What microservices actually are
- When NOT to use microservices
- Service boundaries
- Database per service
- Communication patterns
- Service registry concept
- Deployment independence
- Versioning services
- Debugging microservices
- Trade-offs

Practical

- Break monolith into services
- Define service boundaries
- Setup multiple service servers

Main Project 2

Multi-Service Architecture Simulation

Services:

- Auth service
- Product service
- Order service

- API gateway
-

PART 5 — Pub/Sub & Event-Driven Systems

Core Topics

- What pub/sub is
- Producer vs consumer
- Event-driven architecture
- Async communication
- Event payload design
- Event versioning
- Idempotency
- Event ordering
- Handling failures
- Event replay concept

Practical

- Publish events
- Subscribe services
- Retry failed events
- Log events

Mini Project

Event Bus System (Node + Redis)

- Event publisher
- Event consumer
- Retry logic
- Logging

PART 6 — Message Brokers (Kafka/RabbitMQ Concepts)

Core Topics

- Why message brokers exist
- Queue vs stream
- Kafka basics (conceptual)
- RabbitMQ basics
- Producers/consumers
- Topics & partitions
- Delivery guarantees
- At-least-once delivery
- Exactly-once concept
- Scaling consumers

Practical

- Simulate queue with Redis/BullMQ
- Multiple consumers
- Retry system
- Dead letter queue concept

Mini Lab

Design architecture:

"Order system using queue".

PART 7 — Multi-Tenant Backend Architecture

Core Topics

- What multi-tenant means
- Tenant isolation strategies
- Shared DB vs separate DB
- Tenant middleware
- Tenant-based auth
- Scaling multi-tenant SaaS
- Data partitioning
- Security concerns
- Tenant configs
- SaaS architecture basics

Practical

- Add tenantId to system
- Tenant-based filtering
- Tenant middleware

Main Project 3

Multi-Tenant SaaS Backend

- Multiple organizations
 - Separate data per tenant
 - Role-based access
-

PART 8 — Distributed Caching & Performance Scaling

Core Topics

- Distributed cache concept
- Cache layer in microservices

- Redis cluster concept
- Cache invalidation at scale
- Read-heavy optimization
- Cache stampede
- Write-through caching
- CDN integration concept
- Edge caching basics
- Performance monitoring

Practical

- Cache across services
- Invalidate cache globally
- Benchmark performance

Mini Project

Distributed Cache Layer

- Shared Redis cache
 - Multi-service usage
 - Cache invalidation
-

PART 9 — Resilience & Fault Tolerance

Core Topics

- System failure types
- Circuit breaker pattern
- Retry pattern
- Timeout handling
- Fallback responses

- Bulkhead pattern
- Graceful degradation
- Health checks
- Observability basics
- Chaos testing mindset

Practical

- Simulate service crash
- Implement retry logic
- Add circuit breaker
- Add health check endpoints

Mini Project

Resilient Service Layer

- Circuit breaker
 - Retry logic
 - Timeout handling
 - Fallback response
-

PART 10 — Final Scaling Projects

Final Project 1 — GraphQL Production API

Includes:

- Auth
- Optimized resolvers
- Pagination
- Caching
- Error handling

Final Project 2 — Event-Driven Order System

Includes:

- Order service
 - Payment simulation
 - Event publishing
 - Queue processing
 - Retry logic
 - Logging
-

Final Project 3 — Multi-Service Architecture Simulation

Includes:

- API gateway
 - Auth service
 - Product service
 - Order service
 - Redis cache
 - Event bus
 - Logging
-

▼ MODULE 10 — Database Deep Dive

Goal: make them database engineer-level strong.

After this: they don't "use DB". They design, optimize, and scale it.

PART 1 — Relational Database Foundations (Real Engineering)

Core Topics

- What RDBMS actually does internally
- Tables, rows, pages, storage engine
- Primary keys vs surrogate keys
- Constraints (unique, check, foreign key)
- Data types deeply
- Schema design philosophy
- Entity relationship modeling
- Cardinality
- Referential integrity
- When relational DB shines

Practical

- Design ER diagram
- Implement schema in PostgreSQL
- Enforce constraints
- Test constraint failures

Mini Lab

Design DB for:

- E-commerce
 - SaaS
 - Blog platform
-

PART 2 — Normalization (1NF → 3NF → Beyond)

Core Topics

- Functional dependencies

- 1NF (atomicity)
- 2NF (partial dependency)
- 3NF (transitive dependency)
- BCNF concept
- Denormalization tradeoffs
- Redundancy problems
- Update anomalies
- Delete anomalies
- Real-world trade-offs

Practical

- Normalize messy schema
- Compare normalized vs denormalized
- Fix update anomaly

Mini Project

Schema Refactor Challenge

- Given bad schema
 - Normalize to 3NF
 - Benchmark query clarity
-

PART 3 — Indexing Deep Dive (B-Tree, Hash & Beyond)

Core Topics

- Why indexing exists
- B-tree structure
- How search works in B-tree

- Hash indexes
- Composite indexes
- Covering index
- Index scan vs sequential scan
- When indexes hurt performance
- Index selectivity
- Real-world indexing strategy

Practical

- Create index
- Compare indexed vs non-indexed
- Use EXPLAIN ANALYZE
- Remove bad indexes

Main Project 1

Indexed vs Non-Indexed Benchmark

- Large dataset
 - Query performance comparison
 - Execution plan analysis
-

PART 4 — Query Planning & Execution Plans

Core Topics

- How query planner works
- Cost-based optimization
- Execution plan breakdown
- Sequential scan vs index scan
- Nested loop join

- Hash join
- Merge join
- Sorting cost
- Aggregation optimization
- Query rewriting strategies

Practical

- Analyze execution plans
- Optimize slow query
- Add index & compare
- Rewrite inefficient query

Mini Lab

Optimize 5 slow queries to 5x faster.

PART 5 — Transactions, ACID & Isolation

Core Topics

- ACID properties deeply
- Transaction lifecycle
- Commit vs rollback
- Isolation levels:
 - Read uncommitted
 - Read committed
 - Repeatable read
 - Serializable
- Dirty reads
- Phantom reads

- Write conflicts
- Transaction logs
- When to use transactions

Practical

- Simulate concurrent writes
- Trigger dirty read
- Use transactions safely
- Benchmark with isolation changes

Mini Project

Transaction Safe Banking System

- Transfer funds
 - Rollback on error
 - Concurrency simulation
-

PART 6 — Locks, Concurrency & Performance

Core Topics

- Row-level locks
- Table-level locks
- Deadlocks
- Lock detection
- Lock contention
- Optimistic vs pessimistic locking
- Versioning
- High concurrency design
- Read replicas concept

- Scaling reads

Practical

- Simulate deadlock
 - Fix deadlock
 - Benchmark concurrent requests
 - Implement optimistic locking
-

PART 7 — MongoDB Deep Internals (NoSQL Engineering)

Core Topics

- BSON storage format
- Document modeling deeply
- Embedding vs referencing tradeoffs
- Indexing in MongoDB
- Compound indexes
- Aggregation pipeline
- Query planner in Mongo
- Sharding concept
- Replica sets concept
- When to choose Mongo

Practical

- Create indexes
- Benchmark queries
- Aggregation for reports
- Compare embed vs reference

Mini Project

MongoDB Analytics Module

- Aggregation-based dashboard
 - Indexed filtering
 - Performance comparison
-

PART 8 — Redis Internals & Performance Engineering

Core Topics

- Redis data structures deeply
- In-memory storage mechanics
- RDB vs AOF persistence
- Eviction policies
- Memory optimization
- Redis clustering concept
- Redis for caching vs queue
- Sorted sets for leaderboard
- Pub/Sub internals
- High performance usage

Practical

- Benchmark Redis vs DB
 - Configure eviction
 - Build leaderboard
 - Test persistence mode
-

PART 9 — Elasticsearch Fundamentals (Search Engineering)

Core Topics

- Why traditional DB search fails
- Inverted index concept
- Full-text search basics
- Tokenization
- Analyzers
- Relevance scoring
- Fuzzy search
- Autocomplete
- Search performance
- When to use Elasticsearch

Practical

- Index sample data
- Implement full-text search
- Build autocomplete
- Compare SQL LIKE vs ES search

Main Project 2

Search Engine Module

- Product search
- Full-text search
- Fuzzy match
- Pagination
- Ranking relevance

PART 10 — Final Database Engineering Projects

Final Project 1 — Optimize Slow DB Queries

- Add indexes
- Rewrite queries
- Analyze execution plans
- Improve performance 5–10x

Final Project 2 — Multi-Tenant Database Design

- Shared schema
- Tenant isolation
- Indexed tenant queries
- Scalable design

Final Project 3 — Full Search System

- SQL storage
- Elasticsearch search layer
- Sync DB → search index
- Optimized queries

▼ MODULE 11 — DevOps & Production Engineering

Goal: turn developer → deployment engineer.

After this: they can ship, scale, monitor, and maintain production systems.

PART 1 — DevOps Mindset & Production Architecture

Core Topics

- What DevOps actually means
- Dev vs staging vs production
- Immutable infrastructure concept
- Infrastructure as code (concept)
- Deployment lifecycle
- Blue-green deployment
- Canary deployment
- Rollback strategies
- Zero-downtime deployment
- Production architecture layers

Practical

- Design production architecture diagram
 - Plan deployment pipeline
 - Identify failure points
-

PART 2 — Docker Internals (Not Just “docker build”)

Core Topics

- What containers really are
- Containers vs VMs
- Linux namespaces
- Cgroups
- Union file system
- Docker engine architecture
- Image layers

- Container lifecycle
- Volumes deeply
- Networking basics

Practical

- Inspect image layers
 - Create custom image
 - Mount volumes
 - Inspect container processes
-

PART 3 — Advanced Docker (Multi-Stage & Optimization)

Core Topics

- Writing efficient Dockerfiles
- Multi-stage builds
- Reducing image size
- .dockerignore
- Environment configs
- Build caching
- Secrets handling
- Production image hardening
- Health checks
- Debugging containers

Practical

- Convert app to multi-stage build
- Reduce image size 50%+

- Add health check
- Simulate container crash

Mini Project

Optimized Dockerized Backend

- Multi-stage build
 - Production config
 - Health checks
-

PART 4 — Docker Compose & Multi-Service Setup

Core Topics

- docker-compose architecture
- Multi-container orchestration
- Service dependencies
- Environment variables
- Volumes across services
- Networking between containers
- Scaling services
- Logs in compose
- Debugging multi-container apps
- Local production simulation

Practical

- Setup backend + DB + Redis
- Connect services
- Test scaling
- View logs

Main Project 1

Dockerized Full-Stack App

- Frontend
 - Backend
 - DB
 - Redis
 - Configured via Compose
-

PART 5 — CI/CD Engineering (Real Automation)

Core Topics

- CI vs CD
- Pipeline stages
- Automated testing
- Linting automation
- Build automation
- Deployment automation
- Secrets in CI
- Artifact storage
- Version tagging
- Rollback pipeline

Practical

- Setup pipeline steps
 - Add test stage
 - Add build stage
 - Add deploy stage
-

PART 6 — GitHub Actions Deep Dive

Core Topics

- Workflow files
- Jobs & steps
- Matrix builds
- Environment secrets
- Self-hosted runners concept
- Docker build in CI
- Deployment via CI
- Conditional workflows
- Caching dependencies
- Pipeline optimization

Practical

- Create CI workflow
- Docker build in CI
- Auto deploy to server
- Test failure pipeline

Main Project 2

Production CI/CD Pipeline

- Push → test → build → dockerize → deploy
 - Auto tagging
 - Rollback option
-

PART 7 — Kubernetes Deep Dive (Core Concepts)

Core Topics

- Why Kubernetes exists
- Cluster architecture
- Nodes & control plane
- Pods
- ReplicaSets
- Deployments
- Services (ClusterIP, NodePort, LoadBalancer)
- ConfigMaps & Secrets
- Ingress
- Rolling updates

Practical

- Setup local cluster (kind/minikube)
 - Deploy backend
 - Expose service
 - Rolling update simulation
-

PART 8 — Kubernetes Advanced (Scaling & Helm)

Core Topics

- Horizontal Pod Autoscaler
- Resource limits & requests
- Liveness & readiness probes
- StatefulSets concept
- Persistent volumes
- Helm charts basics

- Namespaces
- Multi-service deployment
- Debugging with kubectl
- Production scaling strategies

Practical

- Auto-scale pods
- Add resource limits
- Deploy via Helm
- Simulate load

Main Project 3

Deploy Full-Stack App on Kubernetes

- Backend deployment
 - DB with persistent volume
 - Redis
 - Ingress routing
 - Auto scaling
-

PART 9 — Logging, Monitoring & Observability

Core Topics

- Observability vs monitoring
- Metrics vs logs vs traces
- Structured logging
- Centralized logging concept
- Health check endpoints
- Monitoring containers

- Alerting systems
- Performance tracking
- Uptime monitoring
- Incident response basics

Practical

- Add health endpoints
 - Collect logs
 - Monitor CPU/memory
 - Simulate failure
-

PART 10 — Production Deployment Architecture (Final Integration)

Final Project 1 — Dockerize Full-Stack App

- Multi-stage build
 - Optimized images
 - Health checks
 - Compose-based environment
-

Final Project 2 — CI/CD Automation

- GitHub Actions pipeline
 - Test stage
 - Docker build
 - Auto deployment
 - Rollback strategy
-

Final Project 3 — Kubernetes Production Deployment

- Deploy cluster
 - Setup Ingress
 - Auto-scale pods
 - Persistent DB
 - Logging & monitoring
 - Zero-downtime updates
-

▼ MODULE 12 — Cloud, Scaling & Distributed Systems

Goal: make learner capable of deploying & scaling real-world systems.

After this: they can design infra for 10 → 1M users.

PART 1 — Cloud Fundamentals & Architecture Thinking

Core Topics

- What cloud actually is
- IaaS vs PaaS vs SaaS
- Regions & availability zones
- Public vs private subnet concept
- Cloud architecture layers
- Stateless system design
- Infra vs application scaling
- Cost vs performance trade-offs
- Multi-region thinking
- Cloud security basics

Practical

- Draw cloud architecture diagram
 - Choose region strategy
 - Plan infra for startup app
-

PART 2 — AWS Core Services Deep Dive

Core Topics

- AWS ecosystem overview
- IAM basics
- EC2 deeply
- Instance types
- Security groups
- Key pairs
- Elastic IP
- S3 basics
- RDS basics
- VPC basics

Practical

- Launch EC2
- Setup security group
- Connect via SSH
- Upload to S3
- Setup RDS

Mini Project

Basic Cloud Backend Deployment

- Backend on EC2
 - DB on RDS
 - Static files on S3
-

PART 3 — Compute Scaling & Instance Strategy

Core Topics

- Instance sizing strategy
- CPU vs memory bound apps
- Vertical scaling deeply
- Horizontal scaling deeply
- Cost optimization
- Spot instances concept
- Reserved instances concept
- Scaling bottlenecks
- Stateless app requirement
- Scaling DB vs app

Practical

- Upgrade instance vertically
- Clone instance horizontally
- Compare performance
- Measure latency

Mini Lab

Vertical vs Horizontal Benchmark

- Same app
- Different scaling approach

- Performance comparison
-

PART 4 — Load Balancing & Traffic Distribution

Core Topics

- Why load balancing needed
- Types of load balancers
- L4 vs L7 load balancers
- Reverse proxy concept
- Sticky sessions
- Health checks
- Failover routing
- Geo routing concept
- SSL termination
- Performance tuning

Practical

- Setup load balancer
- Route traffic to multiple servers
- Fail server → observe switch
- Add health checks

Main Project 1

Load Balanced Backend

- 2+ backend servers
- Load balancer routing
- Health check failover
- SSL termination

PART 5 — Auto Scaling & High Availability

Core Topics

- Auto scaling groups
- Scaling policies
- CPU-based scaling
- Request-based scaling
- Multi-AZ deployment
- High availability architecture
- Self-healing systems
- Rolling deployment
- Failover systems
- Cost-performance balance

Practical

- Setup auto scaling
- Simulate traffic spike
- Auto create instances
- Kill instance → auto replace

Mini Project

Self-Healing Backend System

- Auto scaling group
- Health monitoring
- Automatic recovery

PART 6 — CDN & Edge Performance

Core Topics

- What CDN does
- Edge caching
- Static asset delivery
- Cache invalidation
- Geo latency reduction
- CDN for APIs concept
- Image optimization via CDN
- Security via CDN
- DDoS protection basics
- Performance measurement

Practical

- Setup CDN for frontend
 - Cache static files
 - Measure latency global
 - Invalidate cache
-

PART 7 — Caching Layers & Infra Optimization

Core Topics

- Multi-layer caching
- Browser cache
- CDN cache
- Server cache
- DB cache
- Redis in infra

- Cache invalidation strategies
- Cache TTL strategy
- Read-heavy optimization
- Write-heavy trade-offs

Practical

- Add Redis cache
 - Add CDN cache
 - Compare cached vs uncached
 - Benchmark response times
-

PART 8 — Fault Tolerance & Reliability Engineering

Core Topics

- Failure types
- Single point of failure
- Redundancy design
- Failover systems
- Backup strategies
- Disaster recovery basics
- Multi-region concept
- Graceful degradation
- Health monitoring
- Incident response basics

Practical

- Kill primary server
- Failover to secondary

- Backup & restore DB
- Simulate outage

Mini Project

Fault-Tolerant Architecture

- Backup server
 - Failover routing
 - Recovery strategy
-

PART 9 — Stress Testing & Observability

Core Topics

- Why stress testing
- Load testing vs stress testing
- Throughput vs latency
- Bottleneck detection
- Observability basics
- Metrics collection
- Logging at scale
- Alerting basics
- Performance dashboards
- Capacity planning

Practical

- Stress test API
- Measure RPS
- Detect bottleneck
- Optimize system

Main Project 2

Stress Testing Lab

- Simulate 10k users
 - Measure system response
 - Optimize bottlenecks
-

PART 10 — Final Cloud Scaling Projects

Final Project 1 — Scalable Backend Deployment

- Backend on EC2
 - DB on RDS
 - Redis caching
 - S3 storage
 - CDN integration
-

Final Project 2 — Load-Balanced Production Infra

- Multiple servers
 - Load balancer
 - Auto scaling
 - Health monitoring
-

Final Project 3 — Vertical vs Horizontal Scaling Comparison

- Same backend
- Scale vertically
- Scale horizontally
- Compare:
 - Cost

- Performance
 - Stability
-

▼ MODULE 13 — System Design, Architecture & Capstone

Goal:

make learner capable of designing systems used by **lakhs/millions of users**.

After this: they can sit in system design interview or architect meeting confidently.

PART 1 — System Design Fundamentals (Core Thinking)

Core Topics

- What system design actually is
- Functional vs non-functional requirements
- Scalability vs performance
- Availability vs consistency
- Reliability vs cost
- Latency vs throughput
- Read-heavy vs write-heavy systems
- Stateless vs stateful services
- Real-world architecture layers
- Design thinking framework

Practical

- Convert idea → architecture

- Define system requirements
- Identify bottlenecks

Mini Lab

Design:

- URL shortener (basic)
 - Chat app (basic)
-

PART 2 — Scalability Principles & Patterns

Core Topics

- Vertical vs horizontal scaling
- Load balancing strategies
- Stateless service design
- Database scaling problems
- Caching strategies
- Read replicas
- Queue-based scaling
- CDN usage
- Backpressure handling
- Handling spikes

Practical

- Scale sample backend
 - Add caching layer
 - Add queue for heavy tasks
-

PART 3 — Data Partitioning & Storage Strategy

Core Topics

- Why partition data
- Horizontal partitioning (sharding)
- Vertical partitioning
- Range-based sharding
- Hash-based sharding
- Geo sharding concept
- Multi-tenant DB strategy
- Hot partition problem
- Data migration challenges
- Storage cost vs performance

Practical

- Simulate sharded DB
 - Tenant-based partitioning
 - Compare query performance
-

PART 4 — CAP Theorem & Consistency Models

Core Topics

- CAP theorem deeply
- Consistency
- Availability
- Partition tolerance
- Strong consistency
- Eventual consistency
- Read-after-write consistency

- Trade-offs in real systems
- Choosing consistency model
- Real-world examples

Practical

- Design CP system example
 - Design AP system example
 - Decide trade-offs for product
-

PART 5 — Distributed Systems Fundamentals

Core Topics

- What distributed system is
- Network failures
- Partial failure problem
- Leader election concept
- Replication strategies
- Consensus basics (high level)
- Time synchronization issues
- Distributed locks concept
- Service coordination
- Observability in distributed systems

Practical

- Simulate multi-service system
 - Handle service failure
 - Retry logic implementation
-

PART 6 — Communication Patterns & Async Systems

Core Topics

- Sync vs async communication
- REST vs messaging
- Event-driven architecture
- Pub/sub pattern
- Message queue patterns
- Idempotency
- Event ordering
- Retry handling
- Dead-letter queues
- Event versioning

Practical

- Implement event-based workflow
 - Add retry logic
 - Queue-based processing
-

PART 7 — Caching & Performance Architecture

Core Topics

- Multi-layer caching
- CDN cache
- DB cache
- Cache invalidation strategies
- Cache stampede

- Write-through cache
- Lazy cache
- Performance bottlenecks
- Response optimization
- Scaling read-heavy apps

Practical

- Add multi-layer cache
 - Benchmark performance
 - Fix slow endpoint
-

PART 8 — Real Interview-Level System Design Problems

Topics Covered

Design step-by-step:

- URL shortener
- Instagram feed system
- WhatsApp chat system
- E-commerce system
- Ride booking system
- Notification system
- Video streaming system
- Search system
- Payment system basics
- SaaS multi-tenant system

Each includes:

- Requirements
 - Architecture
 - DB design
 - Scaling strategy
 - Trade-offs
-

PART 9 — Architecture Documentation & Decision Making

Core Topics

- Writing architecture docs
- System diagrams (C4 model)
- API contracts
- Tech decision records
- Trade-off explanation
- Capacity planning basics
- Scaling roadmap
- Failure scenario planning
- Monitoring planning
- Cost estimation basics

Practical

- Write architecture doc
 - Create infra diagram
 - Estimate infra cost
 - Plan scaling roadmap
-

PART 10 — FINAL CAPSTONE (Senior Level)

Build: Production Multi-Tenant SaaS

Must Include

Core System

- Multi-tenant architecture
- Auth (JWT + roles)
- Organization system
- Role-based access

Backend

- REST + GraphQL mix
- Redis caching
- Queue system
- Event-driven modules

Frontend

- Next.js dashboard
- Role-based UI
- Optimized performance

Infra & DevOps

- Dockerized system
- CI/CD pipeline
- Kubernetes deploy
- Cloud hosting

Performance & Scaling

- Caching strategy
- Load-balanced infra
- Horizontal scaling plan
- DB indexing & optimization

Observability

- Logging
- Monitoring
- Error tracking
- Health checks

Documentation

- Architecture diagrams
- API docs
- Scaling strategy
- Deployment guide

▼ MODULE 14 — Enterprise Project I

Scalable E-Commerce Platform (Funded-Startup Level)

Goal:

Build a product that looks like **Series-A startup backend + frontend**.

After this: portfolio becomes stronger than most working devs.

PART 1 — Product Architecture & System Blueprint

System to Build

Full scalable e-commerce platform like:
Amazon (mini) + Shopify backend.

Architecture Decisions

- Microservices-based backend
- Next.js frontend
- API Gateway
- Redis caching
- Queue system
- Cloud deployment
- Kubernetes orchestration

Services Breakdown

- Auth service
- Product service
- Order service
- Payment service
- Inventory service
- Notification service
- Admin analytics service

Deliverables

- HLD (high-level design)
 - LLD (low-level design)
 - DB schema design
 - Service communication diagram
 - Scaling strategy doc
-

PART 2 — Monorepo & Microservice Setup

Core Tasks

- Setup monorepo (TurboRepo/pnpm)
- apps:
 - web
 - gateway
 - services/*
- packages:
 - ui
 - types
 - config
- Shared validation & types
- Docker base setup
- Env config system

Outcome

Clean enterprise repo structure.

PART 3 — Auth & User Service (Security First)

Features

- JWT auth
- Refresh tokens
- Role-based access
- Admin vs user roles
- Secure password hashing
- Session tracking

- Rate-limited login
- OAuth optional

Tech Depth

- Token rotation
- Secure cookies
- Middleware auth
- Service-to-service auth

Deliverable

Production-ready auth service.

PART 4 — Product & Inventory Service

Features

- Product CRUD
- Categories
- Variants (size/color)
- Pricing logic
- Inventory tracking
- Stock updates
- Low stock alerts
- Bulk product import

DB Concepts

- Indexing heavy queries
- Search optimization
- Pagination optimization

Deliverable

High-performance product service.

PART 5 — Order & Payment System

Features

- Cart system
- Order creation
- Payment gateway integration (Stripe/Razorpay)
- Payment webhook handling
- Order status tracking
- Invoice generation
- Refund handling
- Idempotent payment flow

Concepts

- Transaction safety
- Event-driven order flow
- Queue-based confirmation

Deliverable

Real production payment flow.

PART 6 — Real-Time & Notification System

Features

- Real-time order tracking
- WebSocket updates
- Email notifications

- Order status notifications
- Admin alerts
- Queue-based notification system

Tech

- Redis pub/sub
 - WebSockets scaling
 - Background workers
-

PART 7 — Admin Analytics Dashboard

Features

- Revenue analytics
- Sales metrics
- Top products
- User growth
- Order heatmaps
- Inventory reports
- Admin controls

Tech

- Aggregation queries
 - Cached analytics
 - Chart-ready APIs
-

PART 8 — Performance & Caching Layer

Features

- Redis caching
- Product cache
- API response cache
- Cache invalidation strategy
- CDN usage
- Query optimization
- Load testing

Benchmark Goals

- <150ms API response
 - Handle 5k concurrent users
-

PART 9 — DevOps, CI/CD & Cloud Deployment

DevOps

- Dockerize all services
- Docker Compose dev env
- CI/CD pipeline
- GitHub Actions deploy

Kubernetes

- Deploy services
- Ingress routing
- Auto scaling
- Secrets management

Cloud

- AWS deployment

- Load balancer
 - CDN
 - SSL
-

PART 10 — Observability, Testing & Documentation

Observability

- Logging system
- Error tracking
- Metrics collection
- Health endpoints
- Alert system

Testing

- API tests
- Load testing
- Failure simulation

Documentation

Must include:

- Architecture diagrams
 - API docs
 - Scaling plan
 - Deployment guide
 - README like real startup repo
-

▼ MODULE 15 — Enterprise Project II

High-Scale Real-Time Collaboration Platform (Slack × Notion Hybrid)

Goal:

Build something that screams **“this engineer understands scale.”**

This is not a CRUD app. This is concurrency, real-time sync, distributed thinking.

Duration: ~70+ hrs

10 brutal parts.

If executed properly → senior-level portfolio piece.

SYSTEM TO BUILD

A multi-tenant collaboration platform with:

- Workspaces (organizations)
 - Channels
 - Real-time chat
 - Collaborative documents (live editing)
 - Mentions & notifications
 - Search across messages & docs
 - Role-based permissions
 - Admin controls
 - Horizontal scaling
 - Production monitoring
-

PART 1 — System Architecture Blueprint

Deliverables

- High-Level Architecture (HLD)

- Service breakdown
- Communication diagram
- Data flow diagram
- Scaling strategy
- Failure strategy

Architecture Overview

Services

- API Gateway
- Auth Service
- Workspace Service
- Chat Service
- Document Service
- Notification Service
- Search Service

Infrastructure

- Redis (pub/sub + cache)
 - WebSocket cluster
 - PostgreSQL (core DB)
 - Elasticsearch (search)
 - Queue (BullMQ/Kafka concept)
 - Kubernetes cluster
-

PART 2 — Multi-Tenant Workspace System

Features

- Workspace creation
- Invite users
- Role-based permissions:
 - Owner
 - Admin
 - Member
 - Guest
- Tenant isolation
- Organization-level configs

Engineering Concepts

- Tenant middleware
 - Tenant-based DB filtering
 - Row-level isolation
 - Indexed tenant queries
-

PART 3 — Real-Time Messaging Engine

Features

- Channels
- Direct messages
- Typing indicators
- Read receipts
- Message reactions
- Presence system
- Message history pagination

Engineering Concepts

- WebSocket handshake
- Room-based broadcasting
- Redis pub/sub scaling
- Handling reconnects
- Message persistence
- Backpressure handling

Scaling Goal

Handle:

- 10k concurrent socket connections
-

PART 4 — WebSocket Scaling & Distributed Sync

Concepts

- Sticky sessions problem
- Load-balanced WebSockets
- Redis adapter scaling
- Horizontal pod scaling
- Presence synchronization
- Message ordering guarantees
- Retry mechanism
- Rate limiting for sockets

Deliverable

Cluster-ready socket layer.

PART 5 — Collaborative Document Editing (Core Complexity)

Features

- Create/edit documents
- Live multi-user editing
- Cursor tracking
- Auto-save
- Version history
- Conflict handling

Engineering Concepts

- Operational Transformation (conceptual)
- CRDT concept (high level)
- Debounced updates
- Patch-based updates
- Snapshot strategy
- Document versioning

This part alone makes the project insane.

PART 6 — GraphQL API Layer

Features

- Unified GraphQL API
- Workspace queries
- Chat queries
- Document queries
- Subscriptions for real-time
- Role-based field protection

Engineering Concepts

- Schema design
 - Resolvers architecture
 - DataLoader optimization
 - Pagination
 - N+1 avoidance
 - Auth in GraphQL
-

PART 7 — Search System (Message + Docs)

Features

- Full-text search
- Fuzzy matching
- Channel-based search
- Workspace-scoped search
- Highlight matches
- Pagination

Tech

- Elasticsearch
 - Index sync system
 - Async indexing via queue
-

PART 8 — Distributed Caching & Performance

Caching Layers

- Redis message cache
- Workspace cache
- Search cache

- Rate limit cache

Performance Goals

- Chat latency < 100ms
- API response < 200ms
- Scalable to 50k users

Concepts

- Cache invalidation
 - Cache stampede prevention
 - Write-through cache
-

PART 9 — DevOps, Scaling & Observability

Infrastructure

- Dockerized services
- Kubernetes deployment
- Horizontal scaling
- Load balancer
- Auto scaling

Observability

- Structured logs
- Health checks
- Metrics collection
- CPU/memory monitoring
- Failure alerts

Stress Testing

- Simulate 5k users chatting
 - Monitor system under load
 - Optimize bottlenecks
-

PART 10 — Final Deliverables & Documentation

Must Include

1. System Design Document

- HLD
- LLD
- Scaling plan
- Data partitioning
- Trade-off decisions
- Cost estimation

2. Architecture Diagrams

- Service communication
- DB schema
- Event flow
- WebSocket cluster diagram

3. Deployment Guide

- Setup instructions
- CI/CD process
- Kubernetes deploy steps

4. Live Deployment

- Cloud-hosted system

- Load-balanced
 - Auto-scaled
-